

Obliczenia Naukowe

Rafał Wochna 279752

0 Opis problemu i wykorzystanej struktury

Struktura macierzy i definicje

W zadaniu potrzebujemy rozwiązać układ równań liniowych

$$Ax = b,$$

gdzie $A \in \mathbb{R}^{n \times n}$ ($n \geq 4$) jest macierzą rzadką o blokowej strukturze:

$$A = \begin{pmatrix} A_1 & C_1 & 0 & \cdots & 0 & 0 \\ 0 & B_2 & A_2 & C_2 & \cdots & 0 \\ 0 & 0 & B_3 & A_3 & C_3 & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & B_{v-1} & A_{v-1} & C_{v-1} \\ 0 & \cdots & 0 & 0 & B_v & A_v \end{pmatrix},$$

gdzie:

- $v = n/\ell$, $\ell \geq 2$ (liczba bloków),
- 0 – macierz zerowa stopnia ℓ .

Definicje bloków

1. Macierze A_k (gęste): $A_k \in \mathbb{R}^{\ell \times \ell}$ dla $k = 1, \dots, v$, postaci:

$$A_k = \begin{pmatrix} a_{11}^{(k)} & a_{12}^{(k)} & \cdots & a_{1\ell}^{(k)} \\ a_{21}^{(k)} & a_{22}^{(k)} & \cdots & a_{2\ell}^{(k)} \\ \vdots & \vdots & \ddots & \vdots \\ a_{\ell 1}^{(k)} & a_{\ell 2}^{(k)} & \cdots & a_{\ell \ell}^{(k)} \end{pmatrix}.$$

2. Macierze B_k (specjalne): $B_k \in \mathbb{R}^{\ell \times \ell}$ dla $k = 2, \dots, v$, postaci:

$$B_k = \begin{pmatrix} b_{1,1}^k & \cdots & b_{1,\ell-2}^k & b_{1,\ell-1}^k & b_{1,\ell}^k \\ 0 & \cdots & 0 & 0 & b_{2,\ell}^k \\ \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & \cdots & 0 & 0 & b_{\ell,\ell}^k \end{pmatrix}.$$

Tylko pierwszy wiersz i ostatnia kolumna są niezerowe.

3. Macierze C_k (diagonalne): $C_k \in \mathbb{R}^{\ell \times \ell}$ dla $k = 1, \dots, v - 1$, postaci:

$$C_k = \begin{pmatrix} c_1^k & 0 & \cdots & 0 \\ 0 & c_2^k & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & c_\ell^k \end{pmatrix}.$$

Wykorzystana struktura danych

Do przechowywania macierzy A o strukturze blokowej zaimplementowałem dedykowany typ `BlockMatrix` w języku Julia, który wykorzystuje format rzadki `SparseMatrixCSC` do efektywnego pamiętania tylko elementów niezerowych oraz dodatkowych elementów zerowych, do których będziemy się odwoływać.

Struktura `BlockMatrix`

Struktura składa się z trzech pól:

- `n::Int` – pełny rozmiar macierzy $n \times n$
- `l::Int` – rozmiar pojedynczego bloku ℓ
- `A::SparseMatrixCSC{Float64, Int64}` – macierz w formacie skompresowanej kolumnowej macierzy rzadkiej (Compressed Sparse Column)

Konieczność jawnej inicjalizacji zer

Jawna inicjalizacja zer konieczna jest w następujących miejscach:

- Pod przekątną w blokach diagonalnych C_k .
- W blokach B_k poza pierwszym wierszem i ostatnią kolumną.
- W blokach diagonalnych C_k , w $\ell - 1$ kolumn na prawo od przekątnej.

Metody pomocnicze

Dla efektywnego przetwarzania macierzy zaimplementowałem funkcje określające zakresy niezerowych elementów w danym wierszu/kolumnie:

```
first_column(mat, i) # pierwsza potencjalnie niezerowa kolumna w wierszu i
last_column(mat, i)  # ostatnia potencjalnie niezerowa kolumna w wierszu i
first_row(mat, j)    # pierwszy potencjalnie niezerowy wiersz w kolumnie j
last_row(mat, j)     # ostatni potencjalnie niezerowy wiersz w kolumnie j
```

1 Metoda eliminacji Gaussa

Metoda eliminacji Gaussa polega ona na sprowadzeniu macierzy A do postaci trójkątnej górnej poprzez eliminację zmiennych, a następnie zastosowaniu podstawienia od dołu do układu równań.

1.1 Proces eliminacji

Dla układu o n równaniach proces składa się z $n - 1$ kroków. W każdym kroku k ($k = 1, \dots, n - 1$):

1. Wybieramy element główny:

- Bez wyboru elementu głównego: Elementem głównym w kroku k jest element diagonalny a_{kk} .
- Z częściowym wyborem elementu głównego: W obrębie kolumny k (dla wierszy $i \geq k$) wybieramy element o największej wartości bezwzględnej:

$$|a_{pk}| = \max_{i \geq k} |a_{ik}|,$$

gdzie p to indeks wiersza z największym elementem. Następnie zamieniamy wiersze k i p (zarówno w macierzy A , jak i w wektorze b).

2. Dla każdego wiersza $i > k$ obliczamy mnożnik:

$$m_{ik} = \frac{a_{ik}}{a_{kk}}.$$

Następnie odejmujemy m_{ik} razy wiersz k od wiersza i :

$$a_{ij} \leftarrow a_{ij} - m_{ik} \cdot a_{kj}, \quad j = k, \dots, n,$$

$$b_i \leftarrow b_i - m_{ik} \cdot b_k.$$

1.2 Wizualizacja procesu eliminacji

Krok 1 ($k = 1$)

Macierz po pierwszym kroku:

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ 0 & a'_{22} & a'_{23} & \cdots & a'_{2n} \\ 0 & a'_{32} & a'_{33} & \cdots & a'_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & a'_{n2} & a'_{n3} & \cdots & a'_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b'_2 \\ b'_3 \\ \vdots \\ b'_n \end{pmatrix}.$$

Krok k (ogólny)

Po $k - 1$ krokach macierz ma postać:

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1,k-1} & a_{1k} & \cdots & a_{1n} \\ 0 & a'_{22} & \cdots & a'_{2,k-1} & a'_{2k} & \cdots & a'_{2n} \\ \vdots & \ddots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & a'_{k-1,k-1} & a'_{k-1,k} & \cdots & a'_{k-1,n} \\ 0 & \cdots & 0 & 0 & a'_{kk} & \cdots & a'_{kn} \\ 0 & \cdots & 0 & 0 & a'_{k+1,k} & \cdots & a'_{k+1,n} \\ \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & 0 & a'_{nk} & \cdots & a'_{nn} \end{pmatrix}.$$

W kroku k eliminujemy zmienną x_k z wierszy $k + 1, \dots, n$.

Krok końcowy ($k = n - 1$)

Po ostatnim kroku otrzymujemy macierz trójkątną górną:

$$U = \begin{pmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \cdots & u_{2n} \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & u_{nn} \end{pmatrix}, \quad \text{wektor prawych stron: } \tilde{b} = \begin{pmatrix} \tilde{b}_1 \\ \tilde{b}_2 \\ \vdots \\ \tilde{b}_n \end{pmatrix}.$$

1.3 Podstawienie od dołu

Mając układ $Ux = \tilde{b}$, rozwiązanie wyznaczamy od ostatniej zmiennej:

$$x_n = \frac{\tilde{b}_n}{u_{nn}},$$

Mając obliczonego x_n możemy obliczyć x_k ze wzoru:

$$x_k = \frac{\tilde{b}_k - \sum_{j=k+1}^n u_{kj}x_j}{u_{kk}}, \quad k = n - 1, \dots, 1.$$

Dzięki temu otrzymamy rozwiązanie układu $Ax = b$

1.4 Złożoność czasowa i pamięciowa dla obu wariantów

Złożoność obliczeniowa eliminacji Gaussa bez wyboru

Klasyczna eliminacja Gaussa ma złożoność $O(n^3)$, ponieważ w każdym kroku eliminacji:

- Należy wyzerować $n - k - 1$ elementów w kolumnie k
- Każde wyzerowanie wymaga odejmowania całych wierszy ($O(n)$ operacji)
- Łącznie daje to $O\left(\sum_{k=1}^{n-1}(n-k)^2\right) = O(n^3)$

Optymalizacja wykorzystująca strukturę blokową Dla macierzy A o strukturze możemy znacząco zredukować tę złożoność:

1. **Ograniczenie liczby elementów do wyzerowania:** W każdej kolumnie pod przekątną znajduje się maksymalnie ℓ elementów niezerowych (związanych z blokami A_k i B_k). Zatem łączna liczba elementów do wyzerowania wynosi $O(\ell \cdot n) = O(n)$ dla stałego ℓ .
2. **Zmniejszenie kosztu odejmowania wierszy:** W kroku k , wiersz zawierający element główny ma:
 - Po lewej stronie tylko zera (dzięki postępowi eliminacji)
 - Po prawej stronie maksymalnie $\ell - 1$ elementów niezerowych (związanych z blokiem C_k)

Odejmowanie takiego wiersza od wierszy poniżej wymaga więc tylko $O(\ell)$ operacji na każdy wiersz, a nie $O(n)$ jak w przypadku ogólnym.

3. **Całkowity koszt eliminacji:** Dla każdego z $n - 1$ kroków:

- Zerowanie ℓ elementów: $O(\ell)$ operacji

$$\text{Całkowita złożoność} = O(n \cdot \ell) = O(n) \quad (\text{dla stałego } \ell)$$

Po eliminacji otrzymujemy układ o macierzy górnotrójkątnej, gdzie w każdym wierszu:

- Na diagonalu znajduje się element niezerowy
- Po prawej stronie znajduje się maksymalnie ℓ elementów niezerowych

Rozwiązanie takiego układu wymaga:

$$O(n \cdot \ell) = O(n) \quad (\text{dla stałego } \ell)$$

Podsumowanie złożoności czasowej

- Eliminacja: $O(n \cdot \ell) = O(n)$ dla stałego ℓ
- Rozwiązanie układu trójkątnego: $O(n \cdot \ell) = O(n)$ dla stałego ℓ
- **Całkowita złożoność:** $O(n)$ dla stałego ℓ

Złożoność czasowa z częściowym wyborem elementu głównego

Częściowy wybór elementu głównego dodaje dodatkowe operacje, ale nie zmienia asymptotycznej złożoności:

1. Wyszukiwanie elementu głównego tj. w każdej kolumnie przeszukujemy maksymalnie ℓ elementów poniżej diagonalu:

$$\text{Koszt na krok} = O(\ell), \quad \text{całkowicie} = O(n \cdot \ell)$$

Całkowita złożoność z częściowym wyborem

$$\text{Całkowity koszt} = O(n \cdot \ell) + O(n \cdot \ell) = O(n \cdot \ell) = O(n) \quad (\text{dla stałego } \ell)$$

Złożoność pamięciowa dla obu przypadków

Macierz A o strukturze przechowujemy efektywnie:

- Liczba elementów niezerowych:

$$\frac{n}{\ell} \cdot \ell^2 + \frac{n}{\ell} \cdot \ell = O(n \cdot \ell) = O(n) \quad (\text{dla stałego } \ell)$$

- Format `SparseMatrixCSC` przechowuje tylko te elementy
- Dodatkowe struktury (wektor permutacji, wektor prawych stron, mnożniki): $O(n)$

Łącznie złożoność pamięciowa: $O(n)$

Pseudokod dla `solve_gauss`

Algorithm 1: `solve_gauss(A, b)`

```
1  $x \leftarrow [0, 0, \dots, 0]$  długości  $n$  ;
2 for  $k \leftarrow 1$  to  $n - 1$  do
3   for  $i \leftarrow k + 1$  to  $\min(k + \ell, n)$  do
4     factor  $\leftarrow A[i, k] / A[k, k]$  ;
5     for  $j \leftarrow k + 1$  to  $\text{last\_column}(A, k)$  do
6        $A[i, j] \leftarrow A[i, j] - \text{factor} \cdot A[k, j]$  ;
7     end
8      $b[i] \leftarrow b[i] - \text{factor} \cdot b[k]$  ;
9   end
10 end
11 for  $i \leftarrow n$  downto 1 do
12   sum  $\leftarrow 0$  ;
13   for  $j \leftarrow i + 1$  to  $\text{last\_column}(A, i)$  do
14     sum  $\leftarrow \text{sum} + A[i, j] \cdot x[j]$  ;
15   end
16    $x[i] \leftarrow (b[i] - \text{sum}) / A[i, i]$  ;
17 end
18 return  $x$ 
```

Pseudokod dla solve_gauss_partial

Algorithm 2: solve_gauss_partial(A , b)

```
1  $x \leftarrow [0, 0, \dots, 0]$  długości  $n$  ;
2  $p \leftarrow [1, 2, \dots, n]$  ;
3 for  $column \leftarrow 1$  to  $n - 1$  do
4    $maxElem \leftarrow 0$  ;
5    $maxIndex \leftarrow column$  ;
6   for  $k \leftarrow column$  to  $last\_row(A, column)$  do
7     if  $|A[p[k], column]| > |maxElem|$  then
8        $maxElem \leftarrow A[p[k], column]$  ;
9        $maxIndex \leftarrow k$  ;
10    end
11  end
12  Zamień  $p[column]$  i  $p[maxIndex]$  ;
13  for  $i \leftarrow column + 1$  to  $last\_row(A, column)$  do
14     $coeff \leftarrow A[p[i], column] / A[p[column], column]$  ;
15    for  $j \leftarrow column + 1$  to  $last\_column(A, column + 1)$  do
16       $A[p[i], j] \leftarrow A[p[i], j] - coeff \cdot A[p[column], j]$  ;
17    end
18     $b[p[i]] \leftarrow b[p[i]] - coeff \cdot b[p[column]]$  ;
19  end
20 end
21 for  $i \leftarrow n$  downto  $1$  do
22    $x[i] \leftarrow b[p[i]]$  ;
23   for  $j \leftarrow i + 1$  to  $last\_column(A, i + 1)$  do
24      $x[i] \leftarrow x[i] - A[p[i], j] \cdot x[j]$  ;
25   end
26    $x[i] \leftarrow x[i] / A[p[i], i]$  ;
27 end
28 return  $x$ 
```

2 Rozkład LU metodą eliminacji Gaussa

Celem jest wyznaczenie rozkładu

$$A = LU,$$

gdzie L jest macierzą dolnotrójkątną z jedynekami na przekątnej, a U macierzą górnortrójkątną. Rozkład jest obliczany metodą eliminacji Gaussa i zapamiętywany efektywnie w macierzy wejściowej A :

- elementy pod przekątną przechowują mnożniki eliminacji m_{ik} (macierz L),
- elementy na przekątnej i powyżej przekątnej przechowują elementy macierzy U .

2.1 Proces rozkładu LU bez wyboru elementu głównego

Dla macierzy A o rozmiarze $n \times n$ proces składa się z $n - 1$ kroków. W każdym kroku k ($k = 1, \dots, n - 1$):

1. Dla każdego wiersza $i > k$ obliczamy mnożnik:

$$m_{ik} = \frac{a_{ik}}{a_{kk}}.$$

2. Zapisujemy mnożnik w miejscu a_{ik} (w macierzy A):

$$a_{ik} \leftarrow m_{ik}.$$

3. Aktualizujemy elementy wiersza i dla kolumn $j = k + 1, \dots, n$:

$$a_{ij} \leftarrow a_{ij} - m_{ik} \cdot a_{kj}.$$

Po zakończeniu algorytmu:

- macierz L dana jest przez

$$l_{ik} = \begin{cases} a_{ik}, & i > k, \\ 1, & i = k, \\ 0, & i < k, \end{cases}$$

- macierz U dana jest przez

$$u_{ik} = \begin{cases} a_{ik}, & i \leq k, \\ 0, & i > k. \end{cases}$$

2.2 Wizualizacja procesu rozkładu LU

Niech początkowa macierz $A^{(0)} = A$.

Krok 1 ($k = 1$)

Obliczamy mnożniki dla $i = 2, \dots, n$:

$$m_{i1} = \frac{a_{i1}^{(0)}}{a_{11}^{(0)}}.$$

Zapisujemy je w pierwszej kolumnie i aktualizujemy wiersze:

$$A^{(1)} = \begin{pmatrix} a_{11}^{(0)} & a_{12}^{(0)} & a_{13}^{(0)} & \cdots & a_{1n}^{(0)} \\ m_{21} & a_{22}^{(1)} & a_{23}^{(1)} & \cdots & a_{2n}^{(1)} \\ m_{31} & a_{32}^{(1)} & a_{33}^{(1)} & \cdots & a_{3n}^{(1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ m_{n1} & a_{n2}^{(1)} & a_{n3}^{(1)} & \cdots & a_{nn}^{(1)} \end{pmatrix},$$

gdzie

$$a_{ij}^{(1)} = a_{ij}^{(0)} - m_{i1} \cdot a_{1j}^{(0)}, \quad i, j \geq 2.$$

Krok k (ogólny)

Po $k - 1$ krokach macierz ma postać:

$$A^{(k-1)} = \begin{pmatrix} u_{11} & u_{12} & \cdots & u_{1,k-1} & u_{1k} & \cdots & u_{1n} \\ m_{21} & u_{22} & \cdots & u_{2,k-1} & u_{2k} & \cdots & u_{2n} \\ \vdots & \ddots & \ddots & \vdots & \vdots & \ddots & \vdots \\ m_{k-1,1} & \cdots & m_{k-1,k-2} & u_{k-1,k-1} & u_{k-1,k} & \cdots & u_{k-1,n} \\ m_{k1} & \cdots & m_{k,k-2} & m_{k,k-1} & a_{kk}^{(k-1)} & \cdots & a_{kn}^{(k-1)} \\ m_{k+1,1} & \cdots & m_{k+1,k-2} & m_{k+1,k-1} & a_{k+1,k}^{(k-1)} & \cdots & a_{k+1,n}^{(k-1)} \\ \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ m_{n1} & \cdots & m_{n,k-2} & m_{n,k-1} & a_{nk}^{(k-1)} & \cdots & a_{nn}^{(k-1)} \end{pmatrix}.$$

W kroku k :

1. Obliczamy mnożniki dla $i = k + 1, \dots, n$:

$$m_{ik} = \frac{a_{ik}^{(k-1)}}{a_{kk}^{(k-1)}}.$$

2. Zapisujemy m_{ik} w miejscu $a_{ik}^{(k-1)}$.
3. Aktualizujemy elementy dla $i > k$ i $j > k$:

$$a_{ij}^{(k)} = a_{ij}^{(k-1)} - m_{ik} \cdot a_{kj}^{(k-1)}.$$

Krok końcowy ($k = n - 1$)

Po ostatnim kroku otrzymujemy macierz zawierającą rozkład LU :

$$A^{(n-1)} = \begin{pmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ m_{21} & u_{22} & \cdots & u_{2n} \\ \vdots & \ddots & \ddots & \vdots \\ m_{n1} & m_{n2} & \cdots & u_{nn} \end{pmatrix}.$$

2.3 Rozkład LU z częściowym wyborem elementu głównego

W tym wariantcie w każdym kroku k :

1. Znajdujemy indeks $p \geq k$ taki, że

$$|a_{pk}| = \max_{i \geq k} |a_{ik}|.$$

2. Zamieniamy wiersze k i p w macierzy A (logicznie, za pomocą wektora permutacji p).

3. Kontynuujemy jak w wariancie bez wyboru, ale używamy wiersza p jako wiersza głównego.

Wynikiem jest rozkład

$$pA = LU,$$

gdzie p jest wektorem permutacji.

2.4 Złożoność czasowa i pamięciowa dla obu wariantów

Złożoność pamięciowa

Rozkład LU jest przechowywany in-place w macierzy A :

- brak dodatkowych macierzy na L i U ,
- opcjonalnie wektor permutacji P o rozmiarze $O(n)$ (dla częściowego wyboru),
- dodatkowo pamięć: $O(n)$ dla macierzy.

Ostatecznie złożoność pamięciowa wynosi $O(n)$ dla obu przypadków.

Złożoność czasowa

Dla macierzy o specyficznej strukturze, w której w każdej kolumnie występuje co najwyżej ℓ elementów niezerowych pod przekątną:

- eliminacja: $O(n \cdot \ell)$,
- podstawienia (rozwiązanie układów $Ly = b$, $Ux = y$): $O(n \cdot \ell)$,

co daje łączną złożoność:

$$O(n) \quad \text{dla stałego } \ell.$$

Pseudokod dla compute_lu (bez wyboru)

Algorithm 3: compute_lu(A)

```

1  $n \leftarrow \text{liczba\_wierszy}(A)$  ;
2 for  $k \leftarrow 1$  to  $n - 1$  do
3   for  $i \leftarrow k + 1$  to  $\min(k + \ell, n)$  do
4      $A[i, k] \leftarrow A[i, k] / A[k, k]$  for  $j \leftarrow k + 1$  to  $\text{last\_column}(A, k)$  do
5        $A[i, j] \leftarrow A[i, j] - A[i, k] \cdot A[k, j]$  ;
6     end
7   end
8 end
9 return  $LU$ 
```

Pseudokod dla compute_lu_partial (z częściowym wyborem)

Algorithm 4: compute_lu_partial(A)

```
1  $n \leftarrow$  liczba wierszy( $A$ ) ;
2  $p \leftarrow [1, 2, \dots, n]$  ; // wektor permutacji
3 for  $k \leftarrow 1$  to  $n - 1$  do
    // szukanie elementu głównego
4      $p_{\max} \leftarrow k$  ;
5      $v_{\max} \leftarrow |A[P[k], k]|$  ;
6     for  $i \leftarrow k + 1$  to  $\min(k + \ell, n)$  do
7         if  $|A[P[i], k]| > v_{\max}$  then
8              $v_{\max} \leftarrow |A[P[i], k]|$  ;
9              $p_{\max} \leftarrow i$  ;
10        end
11    end
12    if  $p_{\max} \neq k$  then
13        zamień  $P[k]$  z  $P[p_{\max}]$  ;
14    end
    // eliminacja
15    for  $i \leftarrow k + 1$  to  $\min(k + \ell, n)$  do
16         $A[P[i], k] \leftarrow A[P[i], k] / A[P[k], k]$  ;
17        for  $j \leftarrow k + 1$  to  $\text{last\_column}(A, k + \ell)$  do
18             $A[P[i], j] \leftarrow A[P[i], j] - A[P[i], k] \cdot A[P[k], j]$  ;
19        end
20    end
21 end
22 return  $LU, p$ 
```

3 Rozwiązywanie układu równań z rozkładem LU

Po wyznaczeniu rozkładu $A = LU$ (lub $pA = LU$ z częściowym wyborem elementu głównego), rozwiązanie układu $A\mathbf{x} = \mathbf{b}$ sprowadza się do rozwiązania dwóch układów trójkątnych.

3.1 Rozwiązywanie bez permutacji

Dla rozkładu $A = LU$:

1. Rozwiązujemy układ $L\mathbf{y} = \mathbf{b}$ (podstawienie w przód)
2. Rozwiązujemy układ $U\mathbf{x} = \mathbf{y}$ (podstawienie od dołu)

Układ $L\mathbf{y} = \mathbf{b}$, gdzie L jest dolnotrójkątna z jedynkami na przekątnej:

$$\begin{pmatrix} 1 & 0 & \cdots & 0 \\ l_{21} & 1 & \cdots & 0 \\ \vdots & \ddots & \ddots & \vdots \\ l_{n1} & l_{n2} & \cdots & 1 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix}.$$

Podstawienie w przód ($i = 1, \dots, n$):

$$y_i = b_i - \sum_{j=1}^{i-1} l_{ij}y_j.$$

Następnie układ $U\mathbf{x} = \mathbf{y}$:

$$\begin{pmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \cdots & u_{2n} \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & u_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}.$$

Podstawienie od dołu ($i = n, \dots, 1$):

$$x_i = \frac{y_i - \sum_{j=i+1}^n u_{ij}x_j}{u_{ii}}.$$

3.2 Rozwiązywanie z permutacją (częściowy wybór elementu głównego)

Dla rozkładu $PA = LU$, gdzie P jest macierzą permutacji:

1. Rozwiązujemy układ $L\mathbf{y} = P\mathbf{b}$ (podstawienie w przód z permutacją)
2. Rozwiązujemy układ $U\mathbf{x} = \mathbf{y}$ (podstawienie od dołu)

Niech p będzie wektorem permutacji reprezentującym p . Wtedy:

$$p\mathbf{b} = [b_{p_1}, b_{p_2}, \dots, b_{p_n}]^T.$$

Układ $L\mathbf{y} = p\mathbf{b}$ rozwiązujemy podobnie jak w wariancie bez permutacji, ale używając przestawionych elementów wektora $\mathbf{b}$.

3.3 Złożoność pamięciowa i czasowa dla obu wariantów

Złożoność pamięciowa

Dla obu wariantów rozwiązywania układu równań z rozkładem LU:

- Macierz LU : rozkład przechowywany *in-place* w macierzy A , zajmując tyle samo pamięci co macierz wejściowa

- Wektor rozwiązania $\mathbf{x}$: $O(n)$
- Wektor permutacji p : $O(n)$ (tylko w wariancie z częściowym wyborem elementu głównego)
- Łączna złożoność pamięciowa:
 - Bez permutacji: $O(n \cdot \ell + n) = O(n)$ dla stałego ℓ
 - Z permutacją: $O(n \cdot \ell + 2n) = O(n)$ dla stałego ℓ

Złożoność czasowa

Rozwiązanie układu równań z wykorzystaniem rozkładu LU składa się z dwóch etapów:

1. Podstawienie w przód: rozwiązanie układu $L\mathbf{y} = \mathbf{b}$ (lub $L\mathbf{y} = p\mathbf{b}$)

Dla każdego wiersza $i = 1, \dots, n$:

- Przeglądamy kolumny od `first_column(LU, i)` do $i - 1$
- Dla macierzy o strukturze blokowej: maksymalnie ℓ elementów w każdym wierszu
- Koszt: $O(n \cdot \ell)$

2. Podstawienie od dołu: rozwiązanie układu $U\mathbf{x} = \mathbf{y}$

Dla każdego wiersza $i = n, \dots, 1$:

- Przeglądamy kolumny od $i + 1$ do `last_column(LU, i)`
- Dla macierzy o strukturze blokowej: maksymalnie ℓ elementów w każdym wierszu
- Koszt: $O(n \cdot \ell)$

Podsumowanie złożoności czasowej:

- Podstawienie w przód: $O(n \cdot \ell)$
- Podstawienie od dołu: $O(n \cdot \ell)$
- Łącznie: $O(n \cdot \ell) = O(n)$ dla stałego ℓ

Pseudokod dla solve_lu (bez permutacji)

Algorithm 5: solve_lu(LU, b)

```
1  $n \leftarrow \text{liczba wierszy}(LU)$  ;  
   // Podstawienie w przód:  $Ly = b$   
2 for  $i \leftarrow 1$  to  $n$  do  
3   for  $j \leftarrow \text{first\_column}(LU, i)$  to  $i - 1$  do  
4      $b[i] \leftarrow b[i] - LU[i, j] \cdot b[j]$  ;  
5   end  
6 end  
  
   // Podstawienie od dołu:  $Ux = y$   
7 for  $i \leftarrow n$  downto 1 do  
8   for  $j \leftarrow i + 1$  to  $\text{last\_column}(LU, i)$  do  
9      $b[i] \leftarrow b[i] - LU[i, j] \cdot b[j]$  ;  
10  end  
11   $b[i] \leftarrow b[i] / LU[i, i]$  ;  
12 end  
13 return b ;           // wektor b zawiera teraz rozwiązanie x
```

Pseudokod dla solve_lu_partial (z permutacją)

Algorithm 6: solve_lu_partial(LU, p, b)

```
1  $n \leftarrow \text{liczba wierszy}(LU)$  ;  
2  $x \leftarrow [0, 0, \dots, 0]$  długości  $n$  ;           // tworzymy nowy wektor  
   // Podstawienie w przód:  $Ly = Pb$   
3 for  $i \leftarrow 1$  to  $n$  do  
4    $x[i] \leftarrow b[p[i]]$  ;           // stosujemy permutację:  $Pb$   
5   for  $j \leftarrow \text{first\_column}(LU, p[i])$  to  $i - 1$  do  
6      $x[i] \leftarrow x[i] - LU[p[i], j] \cdot x[j]$  ;  
7   end  
8 end  
  
   // Podstawienie od dołu:  $Ux = y$   
9  $x[n] \leftarrow x[n] / LU[p[n], n]$  ;  
10 for  $i \leftarrow n - 1$  downto 1 do  
11   for  $j \leftarrow i + 1$  to  $\text{last\_column}(LU, i + \ell)$  do  
12      $x[i] \leftarrow x[i] - LU[p[i], j] \cdot x[j]$  ;  
13   end  
14    $x[i] \leftarrow x[i] / LU[p[i], i]$  ;  
15 end  
16 return x
```

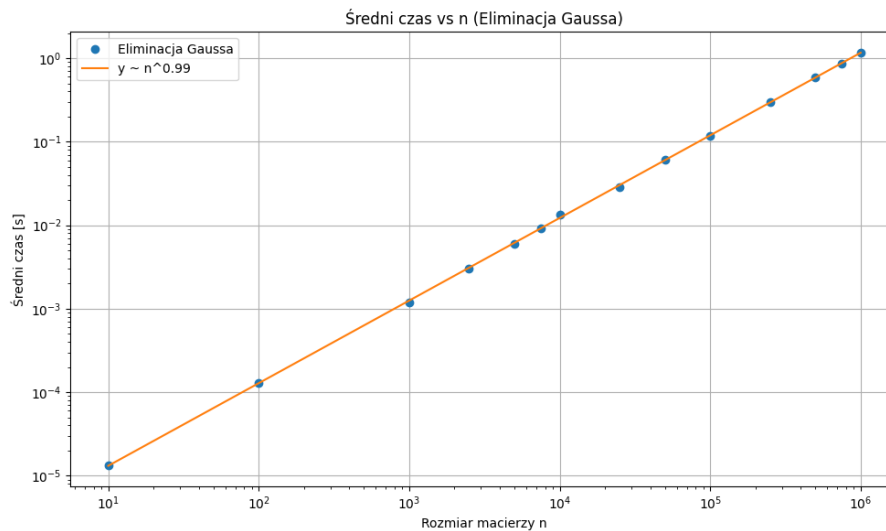
4 Wyniki testów

Metodyka przeprowadzania badań

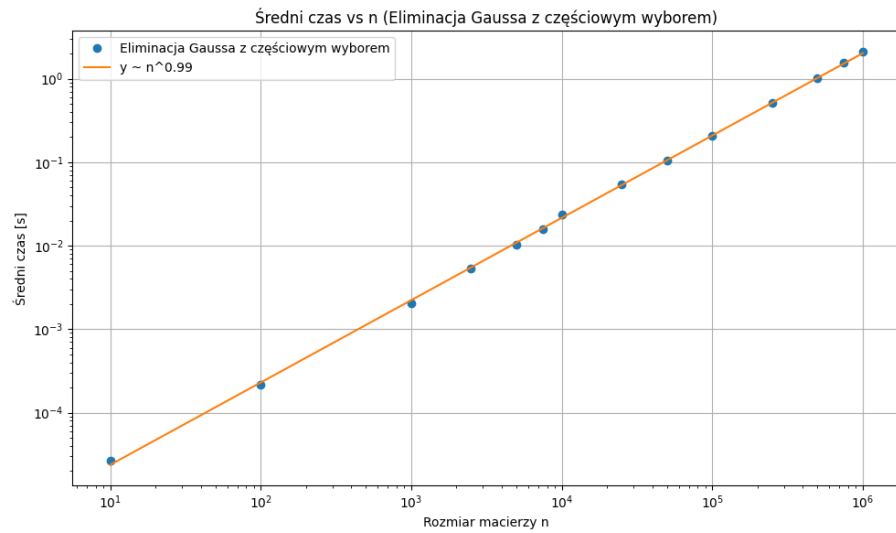
W celu porównania wydajności i dokładności algorytmów rozwiązywania układów równań liniowych, przeprowadziłem badania na 50 losowych macierzach generowanych przy użyciu funkcji `blockmat()`. Testy wykonano dla następujących rozmiarów macierzy: 10, 100, 1000, 2500, 5000, 7500, 10000, 25000, 50000, 100000, 250000, 500000, 750000 oraz 1000000.

Dla każdej macierzy wygenerowano wektor prawej strony b w taki sposób, aby dokładne rozwiązanie było znane. Następnie dla każdego algorytmu obliczono rozwiązanie, a następnie zmierzono czas wykonania, zużycie pamięci (w MB) oraz normę błędu rozwiązania. Wyniki dla każdego rozmiaru macierzy zostały uśrednione po wszystkich 50 losowych przypadkach.

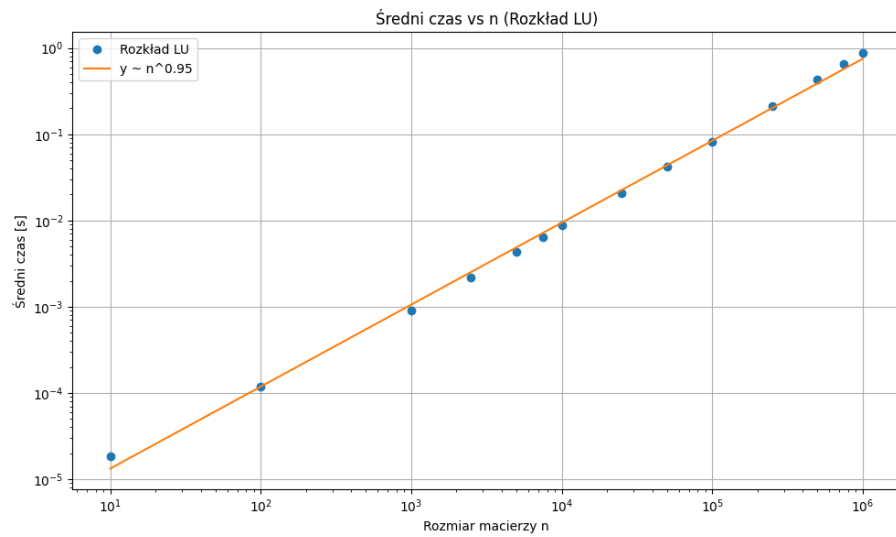
4.1 Czasy działania algorytmów



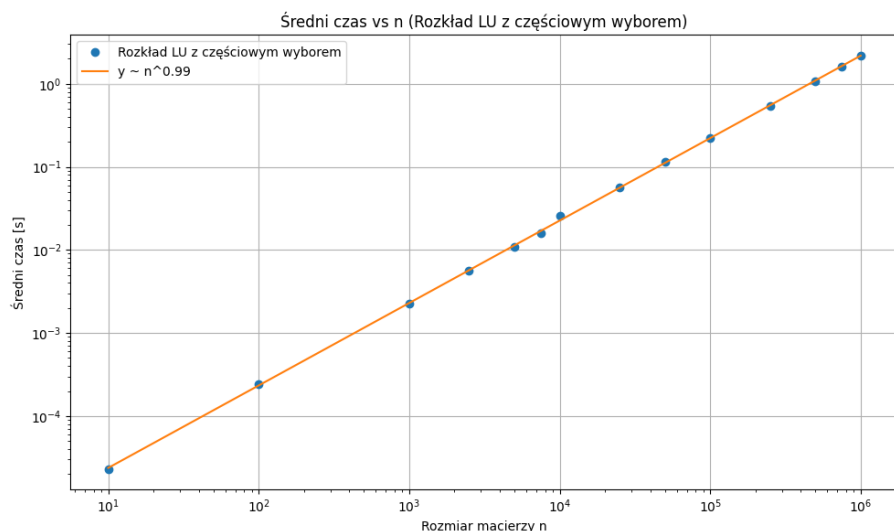
Rysunek 1: Średni czas vs n – Eliminacja Gaussa



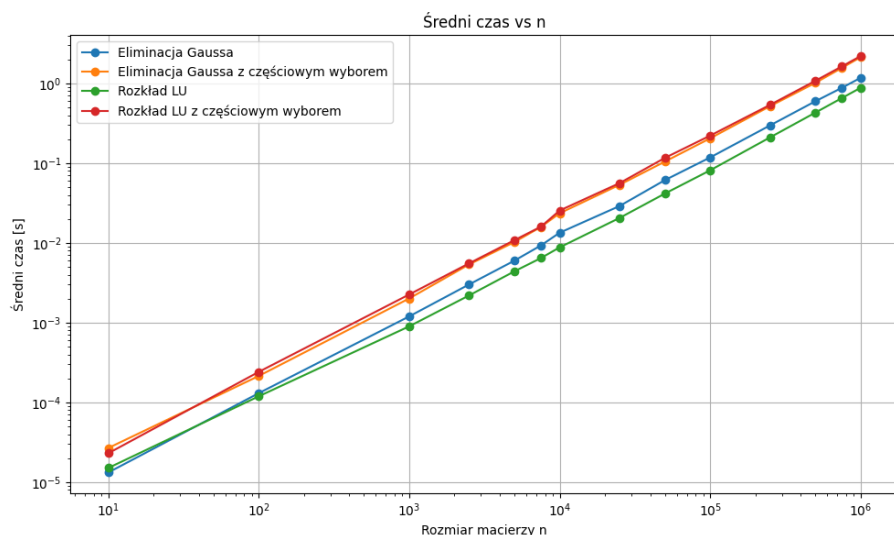
Rysunek 2: Średni czas vs n – Eliminacja Gaussa z częściowym wyborem



Rysunek 3: Średni czas vs n – Rozkład LU



Rysunek 4: Średni czas vs n – Rozkład LU z częściowym wyborem

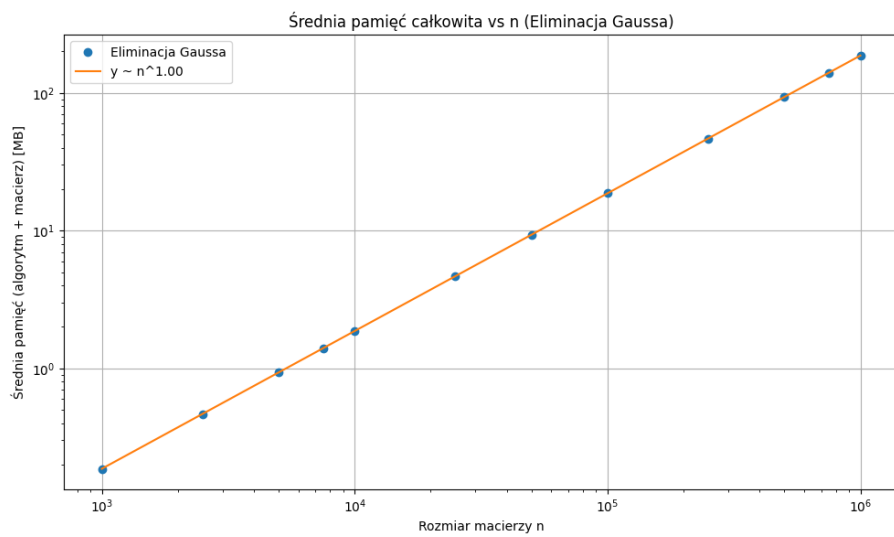


Rysunek 5: Porównanie czasów dla wszystkich algorytmów

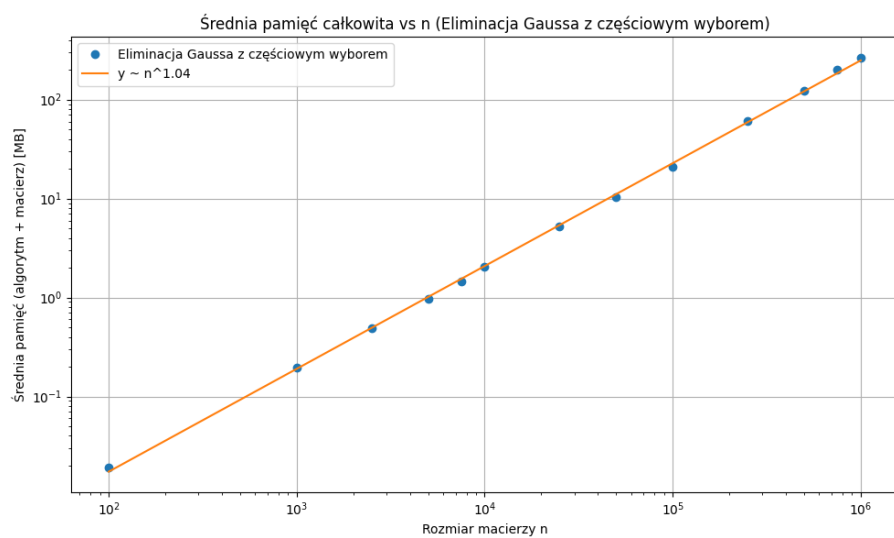
Wnioski i obserwacje dotyczące czasów działania algorytmów

Na podstawie przedstawionych wykresów można zauważyć, że wszystkie analizowane algorytmy wykazują zbliżoną złożoność obliczeniową. Najszybszym algorytmem okazała się rozkład LU bez częściowego wyboru elementu głównego, natomiast najwolniejszy był rozkład LU z częściowym wyborem. Można zaobserwować, także że algorytmy z częściowym wyborem lekko różnią się tylko czasem z algorytmami bez wyboru a jak zobaczymy mają o wiele lepsze wyniki błędu względnego dla coraz większego n . Jesteśmy w stanie zauważyć, że nasza teoretyczna złożoność obliczeniowa $O(n)$ sprawdza się w wynikach obliczeń na wykresach.

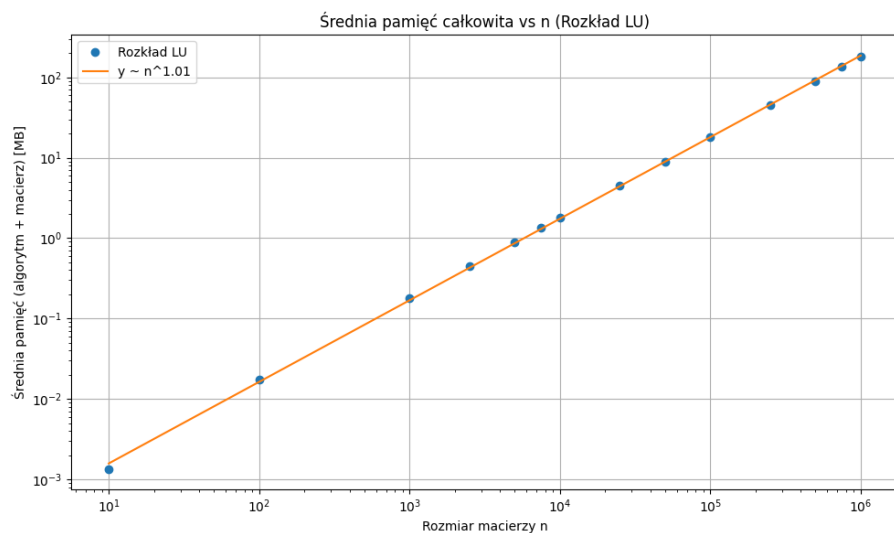
4.2 Zużycie pamięci (macierz + algorytmy)



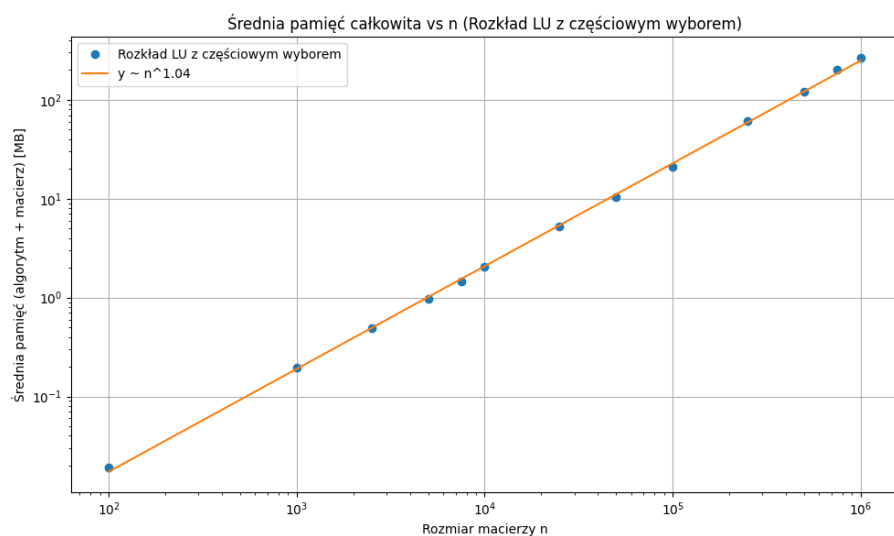
Rysunek 6: Średnia pamięć całkowita vs n – Eliminacja Gaussa



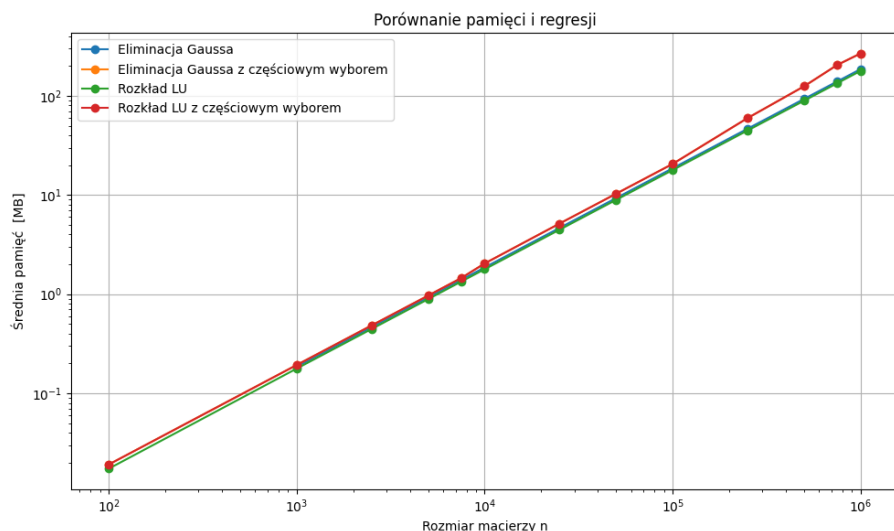
Rysunek 7: Średnia pamięć całkowita vs n – Eliminacja Gaussa z częściowym wyborem



Rysunek 8: Średnia pamięć całkowita vs n – Rozkład LU



Rysunek 9: Średnia pamięć całkowita vs n – Rozkład LU z częściowym wyborem



Rysunek 10: Średnia pamięć potrzebna do przechowania macierzy n

Wnioski i obserwacje dotyczące zużycia pamięci

Analizując wykresy przedstawiające zużycie pamięci przez poszczególne algorytmy, można zauważyć, że wszystkie metody charakteryzują się bardzo zbliżonym, niemal liniowym wzrostem zapotrzebowania na pamięć wraz ze wzrostem rozmiaru macierzy n . Współczynnik potęgowej regresji dla wszystkich algorytmów jest bliski 1, co potwierdza teoretyczną złożoność pamięciową rzędu $O(n)$.

Nieco większe zużycie pamięci można zaobserwować dla algorytmów z częściowym wyborem elementu głównego, co wynika z konieczności przechowywania dodatkowych danych pomocniczych podczas obliczeń. Różnice te są jednak niewielkie w porównaniu do ogólnego trendu wzrostu pamięci.

Podsumowując, wszystkie analizowane algorytmy skalują się bardzo dobrze pod względem zużycia pamięci, a ich implementacje pozwalają na rozwiązywanie układów równań o bardzo dużych rozmiarach.

4.3 Błąd względny

n	Gauss	Gauss_partial	LU	LU_partial
10	$6.000818347 \times 10^{-15}$	$2.255352070 \times 10^{-16}$	$5.406301684 \times 10^{-15}$	$2.255352070 \times 10^{-16}$
100	$1.266568679 \times 10^{-14}$	$2.772033022 \times 10^{-16}$	$1.054095716 \times 10^{-14}$	$2.772033022 \times 10^{-16}$
1000	$3.315267807 \times 10^{-14}$	$2.894417007 \times 10^{-16}$	$2.974552231 \times 10^{-14}$	$2.894417007 \times 10^{-16}$
2500	$4.853544087 \times 10^{-14}$	$2.888766940 \times 10^{-16}$	$6.075723452 \times 10^{-14}$	$2.888766940 \times 10^{-16}$
5000	$1.050250910 \times 10^{-13}$	$2.897063156 \times 10^{-16}$	$6.970843880 \times 10^{-14}$	$2.897063156 \times 10^{-16}$
7500	$1.097397378 \times 10^{-13}$	$2.909390327 \times 10^{-16}$	$8.355246936 \times 10^{-14}$	$2.909390327 \times 10^{-16}$
10000	$3.922319757 \times 10^{-13}$	$2.906578719 \times 10^{-16}$	$7.431696631 \times 10^{-13}$	$2.906578719 \times 10^{-16}$
25000	$1.152317564 \times 10^{-13}$	$2.906449432 \times 10^{-16}$	$1.109626580 \times 10^{-13}$	$2.906449432 \times 10^{-16}$
50000	$1.881926283 \times 10^{-13}$	$2.909773794 \times 10^{-16}$	$1.882026316 \times 10^{-13}$	$2.909773794 \times 10^{-16}$
100000	$2.633017976 \times 10^{-13}$	$2.905126863 \times 10^{-16}$	$3.525456177 \times 10^{-13}$	$2.905126863 \times 10^{-16}$
250000	$4.265056723 \times 10^{-13}$	$2.906382521 \times 10^{-16}$	$3.628359277 \times 10^{-13}$	$2.906382521 \times 10^{-16}$
500000	$3.673675751 \times 10^{-13}$	$2.905926024 \times 10^{-16}$	$4.374509785 \times 10^{-13}$	$2.905926024 \times 10^{-16}$
750000	$7.634504146 \times 10^{-13}$	$2.906814949 \times 10^{-16}$	$6.569887788 \times 10^{-13}$	$2.906814949 \times 10^{-16}$
1000000	$1.327098019 \times 10^{-12}$	$2.907250487 \times 10^{-16}$	$1.850647475 \times 10^{-12}$	$2.907250487 \times 10^{-16}$

Tabela 1: Średnia norma błędu dla różnych algorytmów i rozmiarów macierzy

Wnioski i obserwacje dotyczące jakości obliczeń

Tabela powyżej przedstawia średnie wartości normy błędu uzyskane dla różnych algorytmów rozwiązywania układów równań liniowych oraz różnych rozmiarów macierzy n . Wartości zostały uśrednione po 50 losowych macierzach dla każdego rozmiaru.

Można zauważyć, że algorytmy z częściowym wyborem elementu głównego (**Gauss_partial** oraz **LU_partial**) charakteryzują się znacznie mniejszym błędem numerycznym w porównaniu do ich odpowiedników bez wyboru. Dla wszystkich rozmiarów macierzy norma błędu dla tych algorytmów pozostaje na poziomie rzędu 10^{-16} , co świadczy o bardzo wysokiej dokładności rozwiązań.

W przypadku algorytmów bez częściowego wyboru (**Gauss** oraz **LU**) błąd rośnie wraz ze wzrostem rozmiaru macierzy, osiągając dla największych rozmiarów wartości rzędu 10^{-12} , a nawet 10^{-13} . Oznacza to, że dla dużych układów równań stabilność numeryczna tych metod jest wyraźnie gorsza.

Podsumowując, zastosowanie częściowego wyboru elementu głównego znacząco poprawia dokładność uzyskiwanych rozwiązań, zwłaszcza dla dużych macierzy